

TicTacTUM: Vision-Based TicTacToe Robot

Physical Board Perception, Game-State Reasoning and Robotic Manipulation

Muammar Fadhlan Aldira ,
Gardiyawasam Punchi Hewage Vinul Sanjitha Punchihewa , Nawal Salama ,
Rutendo Courtney Makoni , Mohsin Aejaaz Cahilog Rahman ,
Khaliun Khasagsukh , and Süheyla Caglayan 

 go57zox@tum.de
 vinul.punchichewa@tum.de
 nawal.salama@tum.de
 ge84bar@mytum.de
 mohsin.rahman@mytum.de
 khaliun.khasagsukh@tum.de
 go67naf@mytum.de

September 9, 2026

Abstract — This report presents the design and implementation of a vision-based physical TicTacToe robot as a small cyber-physical system (CPS). A camera observes a real 3×3 game board, computer vision converts the image into a symbolic board state, an unbeatable minimax search selects the robot's move, and an SO-101 robotic arm physically performs robot moves using recorded joint trajectories. The central engineering challenge is not the TicTacToe rules themselves, but the reliable interaction between perception, software state, and physical actuation. The system therefore uses a calibrated perspective transform, HSV-based cell-wise piece classification, minimax move selection, per-cell trajectory playback, and post-action visual verification, all coordinated through a non-blocking, speech-narrated user interface. The report also analyses the kinematics architecture of LeRobot's RobotKinematics implementation using a URDF and the Placo solver. This kinematics layer is treated as a technically grounded extension toward Cartesian target generation; the current project uses per-cell recorded joint-space trajectories selected by the move search. The resulting architecture demonstrates a complete perception–decision–action loop and provides a practical basis for future improvements such as online inverse kinematics, stronger perception, and quantitative closed-loop evaluation.

Keywords: cyber-physical systems, computer vision, robotic manipulation, game-state reasoning, minimax, kinematics

1 Introduction

Traditional TicTacToe software operates entirely in a virtual environment. This project instead treats the game board as a physical world that must be sensed and manipulated. The resulting system combines a camera, image processing, minimax-based game-state reasoning, and a robotic arm in a closed interaction loop. The project repository describes the intended loop as player move, camera detection, game-state evaluation, robot movement, and camera verification [1].

The implemented system uses OpenCV for visual processing and a SO-101 arm controlled through the LeRobot interface. The repository contains the main game program, robot-motion utilities, arm-recording software, and pre-recorded trajectories for every board cell [1]. The physical game is based on an explicit control architecture: perceive the board, determine the current state, select the optimal robot move via minimax, execute the corresponding recorded trajectory, and check the resulting board.

The objectives are:

1. reliably transform camera images into a 3×3 symbolic board representation;
2. separate human interaction from robot action through explicit software states;
3. select an optimal robot move algorithmically and execute the corresponding recorded trajectory at a fixed playback rate;

4. verify that physical actions are reflected in the visual board state; and
5. establish an architecture that can later be extended from recorded joint trajectories to Cartesian and inverse-kinematic control.

2 Requirements and System Overview

2.1 Functional requirements

The system shall:

- capture images of the complete game board;
- calibrate the board region using four corner points;
- rectify the board into a fixed square image;
- classify each of the nine cells as empty, X, or O;
- determine whether the game is ongoing, won, or drawn;
- select an optimal move for the robot and execute the corresponding trajectory;
- keep the camera/UI responsive while a robot trajectory executes; and
- inspect the board again after robot movement.

2.2 Non-functional requirements

The system should be repeatable, understandable, and safe enough to use. Calibration should be persistent, robot trajectories should be validated before execution, and robot communication should be cleaned up after normal completion or interruption. The control loop should also make failures visible rather than silently converting an uncertain camera observation into a physical action.

2.3 Architecture

The architecture is divided into five logical layers:

1. Physical layer: board, pieces and robot;
2. Sensing layer: camera and image acquisition;
3. Perception layer: calibration, perspective correction and cell classification;
4. Decision layer: board representation, minimax move selection, and state machine; and
5. Actuation layer: trajectory loading, robot communication and physical placement.

The separation is valuable because each layer has a different failure mode. Camera problems affect perception; an incorrect board state affects game logic; and communication or trajectory problems affect actuation.

3 Hardware and Robot Design

The physical setup consists of a camera observing the board and a SO-101 arm positioned so that its end effector can place pieces into the relevant cells. The repository identifies the SO-101 as the robotic platform [1]. The software represents six robot joints: shoulder pan, shoulder lift, elbow flex, wrist flex, wrist roll and gripper.

The camera is mounted above the board so that all nine cells are visible. Because the camera is not necessarily normal to the board, direct image coordinates are not suitable for cell segmentation. A four-point projective calibration is therefore performed once and saved as a JSON file. The robot side uses a simpler strategy than full motion planning: a trajectory is recorded in joint space for every board cell and stored as a text file, which reduces online computation to a lookup rather than a real-time solve.

Table 1 Main hardware/software responsibilities.

Element	Responsibility
Camera	Observe physical board and provide frames for perception
Game board	Physical state manipulated by the robot and human player
SO-101 arm	Place the robot's game piece
Host computer	Run vision, minimax logic, UI/TTS and robot communication
LeRobot interface	Provide robot connection and action interface
Trajectory files	Store repeatable joint-space motion sequences per cell

4 Computer Vision

4.1 Calibration and perspective correction

The calibration process uses four manually selected corners in the camera image: top-left, top-right, bottom-right and bottom-left. These points define a planar quadrilateral corresponding to the physical board. OpenCV's perspective transformation then maps this quadrilateral to a fixed 300×300 pixel square in the implementation [1].

If $p = [x, y, 1]^T$ is a point in the camera image, the rectified point p' can be written as

$$s \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad (1)$$

where H is the 3×3 homography estimated from the four point correspondences. This operation makes the board geometry predictable even when the original camera image is skewed.

4.2 Cell segmentation

After warping, the image is divided into nine equal cells, and each cell is inset by one fifth of its size on every side before analysis, which excludes the grid lines themselves from the classification region. Each cell is processed independently, since the expected position of each piece is already known.

The implementation converts each cell from BGR to HSV and applies two purpose-built color masks:

- **X (yellow piece):** Hue [22, 38], Saturation [70, 255], Value [110, 255]. The lower hue bound of 22 excludes dark amber/mustard tones, and the saturation floor of 70 prevents white or near-white regions from being captured.
- **O (black / orangish-grey piece):** the union of a black mask (Hue [0, 180], Saturation [0, 255], Value [0, 75]) and an orangish-grey mask (Hue [5, 25], Saturation [20, 160], Value [60, 180]), combined with a bitwise OR.

Both masks are cleaned with a morphological opening (a 3×3 kernel) to remove small noise. A cell is classified as X or O if the corresponding mask covers more than 12% of the cell area *and* the largest contour in that mask exceeds 40 pixels in area; otherwise the cell is classified as empty.

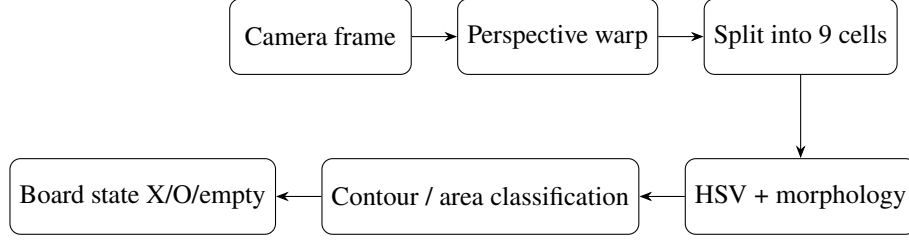


Figure 1 Vision pipeline from camera frame to symbolic board state.

The approach is computationally lightweight but depends on controlled lighting, piece appearance and camera placement. Shadows, reflections, partial occlusion or a displaced camera can cause misclassification, and the fixed hue/saturation/value bounds are tuned to specific piece colors (yellow and black/orangish-grey) rather than learned from data. The calibration file is therefore part of the system configuration rather than an optional visualization parameter.

5 Game Logic and Minimax Move Selection

The software represents the board as nine symbolic cells. A game state can be expressed as

$$B \in \{X, O, \emptyset\}^9. \quad (2)$$

The game-status function evaluates the eight possible winning lines: three rows, three columns and two diagonals. If a line contains three equal non-empty symbols, the corresponding player wins. If there is no winning line and no empty cell remains, the result is a draw; otherwise the game is ongoing. The human may play as either X or O at the start of a game, with the robot automatically assigned the other symbol.

Robot moves are selected, not scripted: an unbeatable depth-scored minimax search evaluates every legal continuation of the current board and returns the move that maximizes the robot's outcome. The value of a terminal board B at search depth d is

$$V(B, d) = \begin{cases} 10 - d & \text{robot wins at depth } d, \\ d - 10 & \text{human wins at depth } d, \\ 0 & \text{draw.} \end{cases} \quad (3)$$

The depth penalty prefers faster wins and slower losses among otherwise equal outcomes. The robot's move is then $\arg \max_i V$ over all empty cells i , computed by recursively minimizing over the opponent's replies and maximizing over the robot's own replies at each ply.

The CPS program uses four explicit timed stages, with durations drawn directly from the implementation:

$$\text{PLAYER} \rightarrow \text{CHECK} \rightarrow \text{ROBOT} \rightarrow \text{CHECK}, \quad (4)$$

looping back to PLAYER until the game ends. The PLAYER stage gives the human time to make a move. The first CHECK stage reads the physical board and evaluates game status. The ROBOT stage runs `get_best_move()` to select a cell via minimax, then plays back the corresponding trajectory (Section 6). The second CHECK stage reads the board again to provide visual feedback after actuation.

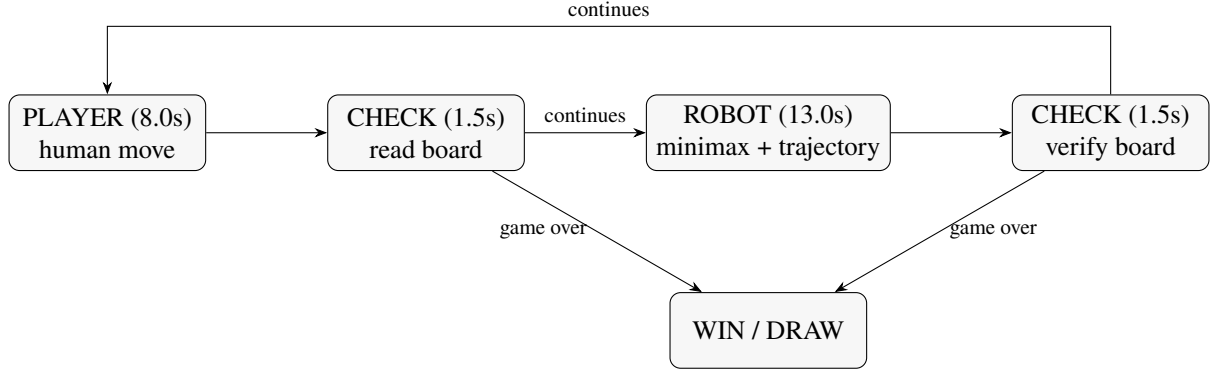


Figure 2 Game-state machine with actual stage durations. Each CHECK evaluates whether the game has ended; only a continuing game proceeds to the next stage, and a completed robot turn returns control to PLAYER for the next round.

This explicit state structure is important because physical actions have duration. The robot cannot be treated as an instantaneous function call: while it is moving, the physical world is changing and the software must remain responsive.

6 Robot Control and Trajectory Playback

The implemented robot controller uses recorded joint trajectories, one per board cell rather than one fixed demonstration sequence. A trajectory contains successive values for the six controlled joints. During playback, the program connects to the SO-101, enables torque, sends joint positions at approximately 20, and disables torque and disconnects during cleanup [1].

A trajectory can be interpreted as

$$q(t_k) = [q_1(t_k), q_2(t_k), q_3(t_k), q_4(t_k), q_5(t_k), q_6(t_k)]^T, \quad (5)$$

where q_6 represents the gripper state. The playback loop sends $q(t_k)$ at fixed intervals. This is a position-command approach rather than a model-based Cartesian controller.

Once minimax returns a target cell index, the controller resolves it to a pair of trajectory files: a start-of-turn trajectory indexed by the robot’s turn count (`o_start_pos_{n}.txt`, $n \in \{1, \dots, 5\}$, since the robot moves at most five times in a nine-cell game), and an end-of-turn placement trajectory indexed by the target cell (`o_end_pos_{cell}.txt`, one per cell, $\text{cell} \in \{1, \dots, 9\}$). This two-part structure lets a single pickup-side trajectory be reused across turns while still placing the piece precisely at whichever cell minimax selects.

The repository also contains a separate arm-recording utility (`record_arm.py`). This enables a practical teach-and-repeat workflow: manually guide the arm, record observations, save the trajectory, and replay the trajectory during the game.

6.1 Concurrency

The main program starts trajectory execution in a separate daemon thread. This allows the camera and interface to remain active while the robot is moving. The architecture therefore separates the time scale of robot actuation from the user-interface loop.

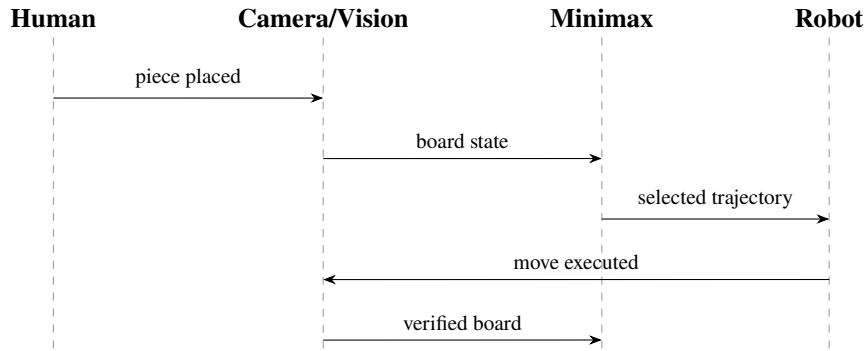


Figure 3 Interaction sequence for a robot turn. Vision is the sole producer of board-state observations, both before and after the robot acts; the minimax module is the sole consumer that decides on and reports the outcome.

7 User Interface and Feedback

The main game runs a Tkinter/OpenCV-driven display with two features beyond a plain camera view. First, a non-blocking speech engine narrates phase transitions – “Player turn”, “Checking board”, “Robot turn” – using a thread-safe queue of fresh `pyttsx3` engine instances per phrase, so speech synthesis never stalls the vision or control loop. Second, an animated background (an optional looping GIF, or a procedurally generated star field as a fallback) runs behind the game UI. Both are cosmetic to the CPS behavior itself but are part of keeping the interface responsive: the animation and speech loop run independently of the camera-read/minimax/trajectory-playback loop described in Section 6, so a slow robot move does not freeze the display.

8 Closed-Loop CPS Operation

The most important CPS property is the feedback loop. The system does not simply calculate a move and assume it happened. Instead:

Physical state → Sensing → Perception → Decision → Actuation → Physical state

For a human turn, the human changes the physical state first. The camera detects this change and updates the software state. For a robot turn, the minimax module selects a target cell and the corresponding trajectory pair changes the physical board. The subsequent CHECK stage then re-reads the entire board and re-evaluates game status.

This feedback perspective is still central to the system’s reliability where it applies: a trajectory completing without communication errors does not necessarily mean the physical game state is correct, and re-reading the board at least catches any mismatch that changes win/draw status.

9 Implementation

The main implementation is organized around a small number of responsibilities. The game program handles camera acquisition, calibration, board perception, minimax move selection, UI/TTS, and robot invocation. A separate motion program supports manual trajectory playback, while a recording program creates new trajectories.

A simplified implementation pattern is:

Table 2 Relevant software modules in the project repository.

Module	Role
main_game.py	Main integrated CPS application (vision, minimax, UI/TTS, robot)
move_robot.py	Standalone trajectory playback
record_arm.py	Manual arm recording
main_game_offline.py	Pygame-only game testing
O_Positions/o_start_pos_*.txt	Per-turn pickup trajectories (5 files)
O_Positions/o_end_pos_*.txt	Per-cell placement trajectories (9 files)
board_corners.json	Persistent camera calibration data

```

frame = camera.read()
board = read_board(frame)
status = check_game_status(board)

if robot_turn(board):
    target_cell = get_best_move(board, robot_symbol, player_symbol)
    execute_trajectory(start_pos[turn_count], end_pos[target_cell])
    verified_board = read_board(camera.read())

```

The actual repository includes additional timing, UI, threading and cleanup logic; the listing above is intentionally a compact representation rather than a reproduction of the source code.

10 Limitations

The main limitation of the current architecture is that robot motion is position-specific. Recorded trajectories are robust when the physical setup is unchanged, but they do not automatically generalize to a moved board or a changed robot mounting. A second limitation is the fixed-threshold color-based vision method (tuned specifically to yellow and black/orangish-grey pieces), appropriate for a controlled demonstrator but less robust than feature-based or learned object detection to lighting changes or different piece colors.

11 Future Work

Several extensions follow naturally from the architecture.

Cartesian control: use camera-to-board calibration and robot kinematics to convert a selected cell into a Cartesian target, then solve IK using the LeRobot/Placo approach, removing the need to record a trajectory for every individual cell.

Extend to other games: the same physical setup could be adapted to play other board games such as chess or checkers.

12 Kinematics and Future Cartesian Control

Note: this section describes a planned extension. It was not implemented or tested in the current system.

Forward kinematics for joint targets q are expressed as:

$${}^wT_E(q) = \begin{bmatrix} R(q) & p(q) \\ 0 & 1 \end{bmatrix}. \quad (6)$$

By utilizing LeRobot’s RobotKinematics interface with Placo [2], target positions for board cells (r, c) can be derived dynamically:

$$p_{rc} = p_0 + r \Delta r + c \Delta c, \quad (7)$$

enabling flexible trajectory generation without requiring pre-recorded joint files.

13 Conclusion

This project demonstrates how a simple game can become a meaningful cyber-physical system when its state exists in the physical world. The system combines camera sensing, geometric calibration, image classification, minimax-based decision making, and robotic actuation. Its main strength is the explicit closed loop: the physical board is observed, converted into a symbolic state, acted upon by an algorithmically-selected move, and observed again.

The current use of recorded joint trajectories is appropriate for a controlled prototype because it minimizes online planning complexity and supports repeatable demonstration, while minimax move selection ensures the robot plays optimally rather than following a fixed sequence. At the same time, the analysis of LeRobot's kinematics interface shows a clear path toward a more general Cartesian controller. A future implementation could map board cells into robot coordinates and use URDF-based forward/inverse kinematics to generate motions dynamically, replacing the per-cell trajectory table entirely.

Overall, the project provides tightly connected perception, computation and physical action. Its limitations also make it a useful engineering platform: improving calibration, perception, motion planning and verification would directly improve the reliability of the complete physical system.

The project source code is available at: <https://github.com/CPSCourse-TUM-HN/TicTacTUM/blob/main/README.md>

14 AI-Generated Content (AIGC) Declaration

In accordance with the academic integrity policy of the course, the authors declare the following use of AI-assisted tools during the preparation of this project and report.

- **Claude (Anthropic)** was used to assist with editing sections of this report, and to support the understanding of technical concepts related to computer vision, robot control, and CPS architecture.
- **Gemini** was used for part of the code debugging and generation.

All AI-generated or AI-assisted content was reviewed, verified, and edited by the authors. The final intellectual responsibility for the design, implementation, analysis, and conclusions presented in this report lies solely with the authors.

References

- [1] Muammar Fadhlan Aldira. *TicTacToe Robot*. <https://github.com/MuammarFadhlanAldira/TicTacToe>. Project repository, accessed September 2026. 2026.
- [2] Hugging Face. *LeRobot: kinematics.py*. <https://github.com/huggingface/lerobot/blob/main/src/lerobot/model/kinematics.py>. Source file accessed September 2026. 2025.